

0.1 MAP Neuron

In this section, we define the MAP neuron. The model is designed for fully online learning (no BPTT), with two practical stabilizers:

1. a sensitivity-gated local update, and
2. an optional sinusoidal phase modulation.

Its stability comes from pre-traces, post-sensitivity gating, low-pass error traces, and optional adaptive threshold dynamics.

Neuron state variables. For hidden neuron i at time t , MAP keeps:

$u_i[t]$ (membrane), $s_i[t] \in \{0, 1\}$ (spike), $a_i[t]$ (adaptive threshold state), $\phi_i[t]$ (phase), $p_j[t]$ (presynaptic trace).

At the output layer, we maintain

$o_c[t]$ (logit trace), $\bar{\delta}_c[t]$ (smoothed output error).

0.1.1 Forward dynamics

Presynaptic trace and membrane update. Given input spikes $x_j[t] \in \{0, 1\}$, the hidden trace and membrane evolve as

$$p_j[t+1] = \lambda_p p_j[t] + x_j[t], \quad (1)$$

$$u_i[t+1] = \alpha u_i[t] + \sum_j w_{ij}^{(1)} x_j[t] + b_i^{(1)} - \beta s_i[t]. \quad (2)$$

Here, $\alpha \in [0, 1)$ is leak and $\beta \geq 0$ is subtractive reset.

Oscillator state. Each hidden neuron has a scalar phase variable that evolves as a discrete-time integrator. At every time step, we first add a constant drift, then optionally couple the phase speed to the current membrane, and finally wrap the phase to one cycle:

$$\phi_i[t+1] = \text{wrap}(\phi_i[t] + \omega_\phi + \kappa_u u_i[t+1]), \quad (3)$$

$$\text{osc}_i[t] = A_\phi \sin(\phi_i[t]). \quad (4)$$

Here, ω_ϕ sets the intrinsic phase increment (hence intrinsic period), A_ϕ sets modulation amplitude, and κ_u controls membrane-to-phase coupling strength.

There are two practical oscillator modes in this implementation:

How oscillator enters computation. We use two modes:

- **learn_only:** oscillator does not alter forward spike decisions. Forward pre-activation remains

$$z_i[t] = u_i[t+1] - \vartheta_i[t],$$

while learning uses an oscillator gain (e.g., $1 + \text{osc}_i[t]$) in the hidden update term.

- **forward_bias:** oscillator is added to forward pre-activation:

$$z_i[t] = u_i[t+1] - \vartheta_i[t] + \text{osc}_i[t].$$

So phase directly shifts instantaneous firing propensity.

If oscillator is disabled, we set $\text{osc}_i[t] \equiv 0$, and both modes reduce to the same baseline dynamics.

Spike pre-activation and firing. Define adaptive threshold and pre-activation:

$$\vartheta_i[t] = \theta_0 + \theta_a a_i[t], \quad (5)$$

$$z_i[t] = u_i[t+1] - \vartheta_i[t] \quad (\text{and if forward_bias: } z_i[t] \leftarrow z_i[t] + \text{osc}_i[t]), \quad (6)$$

$$s_i[t] = \mathbf{1}[z_i[t] \geq 0]. \quad (7)$$

The adaptive threshold state follows

$$a_i[t+1] = \rho_a a_i[t] + \gamma_a (s_i[t] - r_{\text{target}}). \quad (8)$$

This term acts as mild homeostatic regularization: persistent over-firing increases threshold, while under-firing relaxes it.

Algorithm 1 Per-step MAP online update

Require: $x[t], y$, states $\{u, s, a, p, \phi, o, \bar{\delta}\}$

- 1: Update $p[t+1]$ by Eq. (1)
 - 2: Update $u[t+1]$ by Eq. (2)
 - 3: **if** oscillator enabled **then**
 - 4: Update $\phi[t+1]$ and $\text{osc}[t]$ by Eqs. (3)–(4)
 - 5: **end if**
 - 6: Compute $z[t], s[t]$ by Eqs. (6)–(7)
 - 7: Compute $\psi[t]$ by Eq. (9)
 - 8: Update output trace/error by Eqs. (10)–(12)
 - 9: Compute $c[t], \tilde{c}[t]$ by Eqs. (13)–(14)
 - 10: Update $\{W_2, b_2, W_1, b_1\}$ by Eqs. (15)–(18)
 - 11: Update adaptive threshold state $a[t+1]$ by Eq. (8)
-

0.1.2 Online learning rule (LTL form)

Sensitivity gate. MAP uses a local post-sensitivity gate

$$\psi_i[t] = g_\tau(z_i[t]), \quad (9)$$

where g_τ is a surrogate sensitivity (e.g., triangular or sigmoid derivative proxy). Intuitively, when a neuron is in an insensitive region, updates are reduced.

Output trace and smoothed error. For class c :

$$o_c[t+1] = \rho_{\text{out}} o_c[t] + \sum_i w_{ci}^{(2)} s_i[t] + b_c^{(2)}, \quad (10)$$

$$\delta_c[t] = \text{softmax}(o[t])_c - y_c, \quad (11)$$

$$\bar{\delta}_c[t] = \lambda_d \bar{\delta}_c[t-1] + \delta_c[t]. \quad (12)$$

The low-pass error $\bar{\delta}$ damps high-frequency sign flips that are common in online updates.

Local hidden credit and update gate. Hidden local correction signal:

$$c_i[t] = \sum_c (-\bar{\delta}_c[t]) B_{ic}, \quad (13)$$

with fixed feedback matrix B . Define

$$\tilde{c}_i[t] = \begin{cases} c_i[t] \psi_i[t] (1 + \text{osc}_i[t]), & \text{learn_only mode,} \\ c_i[t] \psi_i[t], & \text{otherwise.} \end{cases} \quad (14)$$

Fully online parameter updates. At each step, with minibatch size B_s :

$$\Delta w_{ci}^{(2)}[t] = -\eta_{\text{out}} \frac{1}{B_s} \sum_{b=1}^{B_s} \bar{\delta}_c^{(b)}[t] s_i^{(b)}[t], \quad (15)$$

$$\Delta b_c^{(2)}[t] = -\eta_{\text{out}} \frac{1}{B_s} \sum_{b=1}^{B_s} \bar{\delta}_c^{(b)}[t], \quad (16)$$

$$\Delta w_{ij}^{(1)}[t] = \eta \frac{1}{B_s} \sum_{b=1}^{B_s} \tilde{c}_i^{(b)}[t] p_j^{(b)}[t], \quad (17)$$

$$\Delta b_i^{(1)}[t] = \eta \frac{1}{B_s} \sum_{b=1}^{B_s} \tilde{c}_i^{(b)}[t]. \quad (18)$$

All updates are computed from *current-step* states and traces only; no backward-through-time graph is stored.

0.1.3 Algorithm